

NeutronStar: Dalek's Return

Technical Documentation & System Architecture

Overview: This document provides comprehensive technical documentation for NeutronStar: Dalek's Return, a Java-based real-time strategy game. The system implements resource management, building construction, unit control, and combat mechanics through a modular architecture designed for scalability and maintainability.

System Architecture

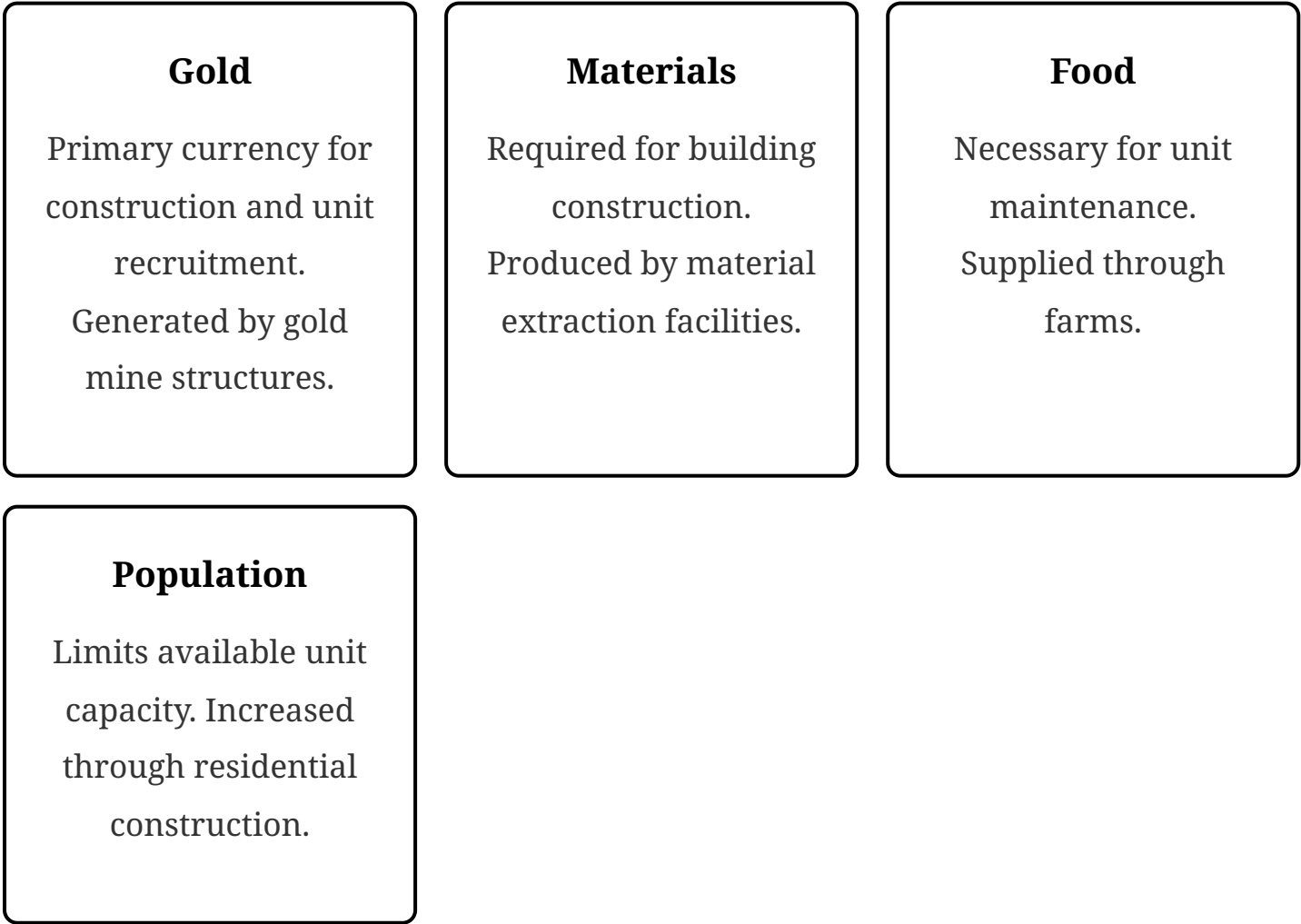
The game engine is built upon seven interconnected core systems, each responsible for specific game functionalities. This modular approach ensures separation of concerns while maintaining efficient inter-system communication through a centralized coordination layer.

Core System Components:

- **EconomySystem** - Manages all economic resources including gold, materials, food, and population metrics
- **BuildingSystem** - Handles building lifecycle from construction initiation through completion and destruction
- **UnitSystem** - Controls unit behavior, movement algorithms, and combat resolution
- **WorldSystem** - Maintains world state including terrain data and spatial entity placement
- **StatsSystem** - Tracks player statistics and calculates popularity metrics
- **InteractionSystem** - Processes user input and translates commands to system actions
- **GameContext** - Provides central coordination and facilitates inter-system communication

Resource Management System

The economy operates on four primary resource types, each serving distinct gameplay functions. Resource generation is managed through passive income mechanisms tied to constructed buildings.



The system implements a delta-based resource calculation model, where each building contributes a per-second increment to the corresponding resource pool. This ensures smooth resource accumulation independent of frame rate variations.

Building System Implementation

The building system implements a streamlined construction workflow that validates prerequisites, manages resource transactions, and applies building effects upon completion.

Construction Workflow

The construction process follows a deterministic sequence: Mode Activation initiates BUILD mode through user input. Building Selection presents available options based on current resources. Placement Validation verifies spatial availability via WorldSystem. Resource Verification confirms sufficient resources through EconomySystem. Transaction Processing deducts resources and initiates construction timer. Completion Handler registers and activates building effects automatically.

Technical Note: Building completion triggers immediate effect registration. A gold mine automatically begins generating 5 gold per second upon construction completion without requiring additional activation commands.

Effect System Architecture

Buildings utilize five modular effect components: GoldEffect, MaterialEffect, FoodEffect, PopulationEffect, and PopularityEffect. Each modifies its corresponding resource or metric. Upon building destruction, all effects are automatically removed to maintain state accuracy.

Unit Control and Combat Systems

Units operate through an AI-driven state machine managing autonomous behavior including idle monitoring, pathfinding, and combat engagement.

State Machine Implementation

Each unit maintains one of four operational states: IDLE performs environmental scanning for threats within detection radius; MOVING executes pathfinding to reach designated coordinates; ATTACKING engages hostile entities within attack range; DEAD indicates unit elimination and removal from gameplay.

Combat Resolution

The combat system implements continuous threat assessment with automatic engagement protocols. The AI controller performs periodic scans for hostile entities within detection range. Upon threat detection, the unit acquires target lock. If the target exceeds attack range, the unit transitions to MOVING state and closes distance. Upon reaching optimal range, it transitions to ATTACKING state and begins damage application. Following target elimination, the unit returns to IDLE state and resumes threat scanning.

Pathfinding Implementation: The system utilizes A* pathfinding algorithm to calculate optimal routes while accounting for terrain cost modifiers. Roads provide minimal traversal cost (1), standard terrain incurs moderate cost (2), and water terrain requires maximum cost (3), ensuring realistic movement patterns.

Economic Model

The economic system operates on an automated resource generation model where building construction establishes passive income streams that function continuously without player intervention.

Economic Development Cycle

A typical progression follows this strategic pattern:

1. Residential construction increases available population capacity
2. Gold mining operations establish primary currency income
3. Material extraction facilities provide construction resources
4. Agricultural development ensures unit sustainability
5. Accumulated resources enable military unit recruitment
6. Military forces engage in autonomous combat operations

Inter-System Communication

The architecture implements a registry pattern to facilitate communication between disparate systems while maintaining loose coupling and modular independence.

Example: Building Construction Process

1. BuildingSystem queries WorldSystem for spatial availability verification
2. BuildingSystem requests resource availability confirmation from EconomySystem
3. EconomySystem executes resource deduction transaction
4. WorldSystem updates spatial occupation data for affected tiles
5. Construction timer progresses over designated time period
6. Upon completion, building effect components register with EconomySystem
7. Resource generation rates are automatically adjusted

Example: Unit-Building Combat Interaction

1. UnitController detects hostile building structure within scan radius
2. Unit assigns building as primary target objective
3. If distance exceeds attack range, MovementSystem calculates path using A* algorithm
4. Unit traverses calculated path while avoiding spatial obstacles
5. Upon reaching effective range, CombatSystem handles damage application
6. Building integrity decreases until destruction threshold is reached
7. Building effects are automatically unregistered from game state

Design Pattern Application: The implementation utilizes a registry pattern to decouple system dependencies. SystemsRegistry provides global access to active system instances, while ServersRegistry maintains building and unit definition templates. This approach ensures modularity while maintaining efficient system communication.

Design Pattern Implementation

The codebase employs several established design patterns to ensure maintainability, scalability, and code organization:

Singleton Pattern

All primary systems (Economy, Building, Unit, World, Stats) are implemented as singleton instances accessible through global registry. This ensures single source of truth for each system's state and eliminates potential duplication issues.

Flyweight Pattern

Building and unit definitions utilize flyweight pattern to minimize memory overhead. A single template definition is created for each entity type and shared across all instances, significantly reducing memory consumption in scenarios with numerous similar entities.

Strategy Pattern

Building effects implement strategy pattern, allowing flexible composition of building behaviors. New building types can be created by combining existing effect strategies or implementing new ones without modifying core system code.

State Machine Pattern

Units implement finite state machine pattern with clearly defined states (IDLE, MOVING, ATTACKING, DEAD) and deterministic transition rules. This approach ensures predictable unit behavior and simplifies debugging processes.

Observer Pattern

InteractionSystem monitors user inputs and dispatches commands to controllers based on operational mode.

Game Loop Architecture

The main game loop executes at approximately 60 frames per second, processing all active systems in a predetermined sequence to ensure consistent state updates:

1. **InteractionSystem** - Processes queued input events and user commands
2. **UnitSystem** - Updates unit states, executes movement calculations, resolves combat
3. **BuildingSystem** - Advances construction timers, processes building destruction events
4. **EconomySystem** - Applies resource generation deltas to current resource pools
5. **StatsSystem** - Recalculates player statistics and popularity metrics

This sequential processing order ensures data consistency and eliminates race conditions between system updates, providing deterministic gameplay behavior across execution cycles.

NeutronStar: Dalek's Return

Technical Documentation • January 2026